

基于改进MD5算法的计算机网络用户身份安全认证方法

连冠

(吉林市第二人民医院, 吉林 吉林 132002)

摘要: 由于计算机网络规模较大、环境复杂, 极易受外界恶意攻击, 导致用户数据泄露等安全问题, 本文提出基于改进 MD5 算法的计算机网络用户身份安全认证方法。将计算机网络用户身份数据加密参量转换为 MD5 算法可处理的格式。改进 MD5 算法迭代过程中的消息扩展算法, 将加密参量代入改进算法中, 定义用户身份认证规则, 进行计算机网络用户身份安全认证。试验结果表明, 设计方法可以有效降低攻击成功率, 并提升吞吐率, 为计算机网络用户身份安全认证提供新的解决方案。

关键词: 改进 MD5 算法; 计算机网络; 用户身份; 身份认证; 安全认证

中图分类号: TP 301

文献标志码: A

DOI:10.13612/j.cnki.cntp.2025.22.040

目前, 计算机网络已渗透生活的方方面面, 带来了便利, 也带来严峻的安全挑战。由于计算机网络具有开放性、匿名性等特征, 用户身份安全认证问题尤为突出, 传统的用户名/密码认证方式存在易被猜测、易泄露等问题, 难以抵御暴力破解、钓鱼攻击等安全威胁。因此, 研究高效的身份安全认证方法成为目前计算机网络安全领域的重要课题。文献[1]针对现有身份认证方法存在的凭证管理繁重、安全性不足等问题, 设计基于区块链和可信执行环境的身份认证方法, 并通过试验验证了该方法的有效性。但是区块链的共识机制和可信执行环境都需要消耗大量的计算资源, 当处理大量身份认证请求时, 这些计算开销可能导致响应速度变慢, 影响用户体验。文献[2]提出基于多级承诺协议的身份认证方法, 具有较高安全性, 但是多级承诺协议通常涉及多轮交互和复杂的加密计算, 增加了节点的计算负担, 导致身份认证耗时较长, 影响整体响应速度。针对上述问题, 本文设计了基于改进 MD5 算法的计算机网络用户身份安全认证方法, 以期提升网络安全水平、保护用户隐私。

1 转换计算机网络用户身份数据加密参量

在计算机网络用户身份认证领域, 应用 MD5 算法进行计算机网络用户身份认证对输入数据长度有严格要求。MD5 算法是一种经典的哈希函数, 其处理机制决定了输入数据必须满足 512 位的整数倍要求。因此, 本文将针对原始计算机网络用户数据加密参量进行转换处理^[3], 将其转换为 MD5 的 512 位标准, 以便后续使用。由于原始计算机网络用户数据的位数不满足 MD5 算法编码需求, 因此本文对其进行补充处理。假设待处理的计算机网络用户数据如公式(1)所示。

$$X=[x_1, x_2, \dots, x_n]^T \quad (1)$$

式中: X 表示计算机网络用户数据集合; n 表示计算机网络用户数据整体字节量; T 表示计算机网络用户数据维度。

在进行数据码数长度补充过程中需要遵循如下规则: 将数据末尾字节作为起始点, 补充字符“1”, 数量为 1, 并在空码数位置上补充字符“0”, 数量与之相对应。基于该规则进行计算机网络用户数据码数补充所得完整数据长度如公式(2)所示。

$$L=(\gamma \times 512 + 448 + 64) \text{ bit} = (\gamma + 1) \times 512 \text{ bit} \quad (2)$$

式中: L 表示补充后所得的完整计算机网络用户数据长度; γ 表示常量。

如公式(2)所示, 本文为了将计算机网络用户数据补充至满足 MD5 算法的完整的 512 位倍数, 将 γ 转换为 64 位码长, 并和原始数据的末尾字节连接在一起。根据上文完成原始计算机网络用户数据码长补充后, 考虑后续改进 MD5 算法对加密参量的特殊需求, 本文将针对补充后数据进行转换处理^[4]。具体来说, 本文定义了一组计算机网络用户数据对接系数, 包括 4 个变量 C_1 、 C_2 、 C_3 和 C_4 , 以此对应用户身份安全认证过程中不同类型的数据加密参量, 各变量的字节形态如公式(3)所示。

$$\begin{cases} C_1 = 0 \times 01234567 \\ C_2 = 0 \times 89abcdef \\ C_3 = 0 \times fedcba98 \\ C_4 = 0 \times 76543210 \end{cases} \quad (3)$$

这些变量在后续的赋值操作中起到关键作用, 代表不同类型数据加密参量的初始状态, 与用户身份认证过程中的各种信息相关联, 例如用户名、密码的某种编码形式等。

基于公式(3)所示 4 个变量 C_1 、 C_2 、 C_3 和 C_4 , 本文参考 MD5 算法的变换规则, 对 4 个变量进行赋值操作, 如公式(4)所示。

$$\begin{cases} C_1 \rightarrow a \\ C_2 \rightarrow b \\ C_3 \rightarrow c \\ C_4 \rightarrow d \end{cases} \quad (4)$$

如公式(4)所示, 经变换赋值操作处理后, 可以得到用于 MD5 算法的 8 个数据基础变量, 将其作为计算机网络用户身份数据加密参量。例如, 在 MD5 算法的某一轮迭代计算中, 假设需要使用基础变量进行运算, 设当前轮的部分中间结果为 M 。根据 MD5 算法逻辑, 进行如公式(5)所示的操作。

$$M = M + a + F(b, c, d) + K \quad (5)$$

式中: F 表示非线性函数; K 表示引入的常数, 用于增加 MD5 算法的随机性。

通过这样的方式, 将转换后的数据对接系数应用于 MD5 算法运算过程中, 将计算机网络用户身份数据加密参量



转换为 MD5 算法可处理的格式。该过程保证了输入数据长度满足改进 MD5 算法的要求, 并进行定义数据对接系数量 and 变换赋值操作, 使其能够参与 MD5 算法的运算, 提升用户身份认证的安全性。

2 改进 MD5 算法安全认证网络用户身份

基于上文获得计算机网络用户身份数据加密参量, 即可应用 MD5 算法进行用户身份安全认证。MD5 算法是一种经典的哈希函数, 可以将任意长度的消息压缩为固定长度的哈希值。由于哈希值具有唯一性和不可逆性, 因此根据哈希值可以进行数据完整性校验、用户身份认证等。但是, 传统 MD5 算法在消息处理机制上存在一定局限性。一般来说, 传统 MD5 算法将输入消息按照 512 比特进行分组, 并对每个分组进行 4 轮循环处理, 每轮循环包括 16 步迭代。在每一轮迭代中, 使用相同的 16 个消息子分组进行计算, 导致算法在消息扩展方面存在局限性, 使攻击者有可能通过构造特定的消息产生碰撞。例如, 攻击者可以精心设计 2 个不同的输入消息, 使它们经传统 MD5 算法处理后产生相同的哈希值, 从而绕过基于 MD5 哈希值的身份认证机制。因此, 针对传统 MD5 算法的消息扩展局限性, 本文借鉴 SHA-1 算法中消息扩展的原则, 引入新的常数与逻辑运算, 改进了 MD5 算法的消息扩展算法^[5], 使消息更难以预测。具体来说, 将输入的计算机网络用户身份数据加密参量按照 512 比特进行分组。如果数据长度不是 512 的整数倍, 那么按照上述数据码数长度补充的方法进行填充, 保证数据长度满足要求。假设原始计算机网络消息子分组为 $X_0, X_2, \dots, X_{15}$, 改进后的计算机网络消息子分组为 $Y_0, Y_2, \dots, Y_{63}$ (由于扩展后消息子分组数量增加, 因此以 64 个为例进行说明, 实际中可根据需要调整)。初次扩展中的表达式为 $Y_i = X_i$, 其中 $i=0, 1, \dots, 15$, 表示前 16 个消息子分组保持不变。进而在后续扩展中, 对 $i=16, 17, \dots, 63$ 采用公式 (6) 进行扩展。

$$Y_i = f_1(Y_{i-2}) + Y_{i-7} + f_2(Y_{i-15}) + Y_{i-16} + K \quad (6)$$

式中: K 表示引入的常数, 用于增加 MD5 算法的随机性。为了进一步增强消息的扩散性, 引入非线性函数 $f_1(x)$ 和 $f_2(x)$ 。

对扩展后的消息子分组进行处理, 如公式 (7) 所示。

$$\begin{cases} f_1(x) = (x \ggg 17) \oplus (x \ggg 19) \oplus (x \ggg 10) \\ f_2(x) = (x \ggg 7) \oplus (x \ggg 18) \oplus (x \ggg 3) \end{cases} \quad (7)$$

式中: $\ggg$ 表示循环右移操作。

在消息子分组处理过程中, 根据不同的迭代轮次和消息子分组索引, 选择合适的非线性函数进行处理, 增强雪崩效应, 使消息中的微小变化能够迅速扩散到整个哈希值中, 从而提高算法的抗碰撞性。最后, 将计算机网络用户身份数据加密参量代入改进后的 MD5 算法。定义用户身份认证规则如下: 1) 哈希值生成。将经过数据转换和消息扩展处理后的计算机网络用户身份数据加密参量输入改进后的 MD5 算法, 经过多轮迭代计算, 生成计算机网络用户哈希值。2) 哈希值比较。将生成的计算机网络用户哈希值与预先存储在数据库中的哈希值进行比较。如果 2 个哈希值一致, 那么说明用户身份合法, 通过身份认证, 允许用户访问网络; 反之, 如果 2 个哈希值不一致, 那么说明用户身份非法, 拒绝用户访

问。

通过上述方式, 改进后的 MD5 算法能够有效实现计算机网络用户身份认证, 保障网络访问的安全性。

3 仿真试验

3.1 试验设置

本节将对所设计的基于改进 MD5 算法的计算机网络用户身份安全认证方法 (试验组) 进行全面、深入的实例数据仿真对比测试。为了更客观、准确地评估试验组方法的性能与效果, 将基于区块链和可信执行环境的计算机网络用户身份安全认证方法和基于多级承诺协议的计算机网络用户身份安全认证方法统一设定为对照组, 分别为对照组 A、对照组 B。在本次试验环境搭建方面, 选用 3 台高性能计算机为硬件基础, 这 3 台计算机均配备了先进的处理器、大容量内存以及高速存储设备, 以保证能够稳定、高效地运行各种身份安全认证方法。每台计算机分别承担运行试验组方法和 2 个对照组方法的任务, 以此保证试验的独立性和可比性。

试验设置 12 组不同的数据量级别, 数据量从 1000 条身份认证请求到 120000 条不等, 这样的数据量级别设置是经过精心考虑的, 以全面模拟不同规模的计算机网络环境。最后, 在试验操作过程中, 在不同数据量级别下, 分别应用试验组方法和 2 个对照组方法, 进行计算机网络用户身份安全认证测试, 并比较测试结果, 以了解基于改进 MD5 算法的计算机网络用户身份安全认证方法在性能、效率和准确性等方面的优势与不足, 同时也能明确其与基于区块链和可信执行环境、基于多级承诺协议的身份安全认证方法间的差异, 为进一步优化和改进身份安全认证技术提供依据。

3.2 结果分析

在本次试验中, 为了验证上述 3 种计算机网络用户身份安全认证方法的安全性, 使用自动化脚本模拟常见的网络攻击, 包括暴力破解、重放攻击等。在每个数据量级别下分别进行 100 次模拟攻击, 统计并整理攻击成功次数, 以计算攻击成功率。在 12 组不同的数据量级别下, 记录试验组和 2 个对照组方法下的计算机网络攻击成功率, 如图 1 所示。

由图 1 可以看出, 在 3 种用户身份安全认证方法下, 虽然计算机网络均存在一定被攻击的风险, 但是试验组方法的安全性显著高于 2 个对照组方法。在试验组方法的用户身份安全认证下, 计算机网络的平均攻击成功率仅为 0.29%, 比 2 个对照组方法分别降低了 0.52%、0.65%。这主要得益于改进 MD5 算法提高了加密复杂性, 当面对大规模网络攻击时, 攻击成功概率下降。由此可以验证, 本文研究的基于改进 MD5 算法的计算机网络用户身份安全认证方法是有效、正确的。

为了进一步验证试验组方法的优越性, 在试验组和 2 个对照组方法进行 12 组不同数据量级别计算机网络用户身份安全认证过程中, 使用网络性能监控工具测量各认证方法的吞吐率, 如图 2 所示。

由图 2 可以看出, 由于随着数据量增加, 认证系统的处理负担增加, 导致吞吐率降低, 因此 3 种计算机网络用户身份安全认证方法的吞吐率均逐渐下降。但是, 与 2 个对照组

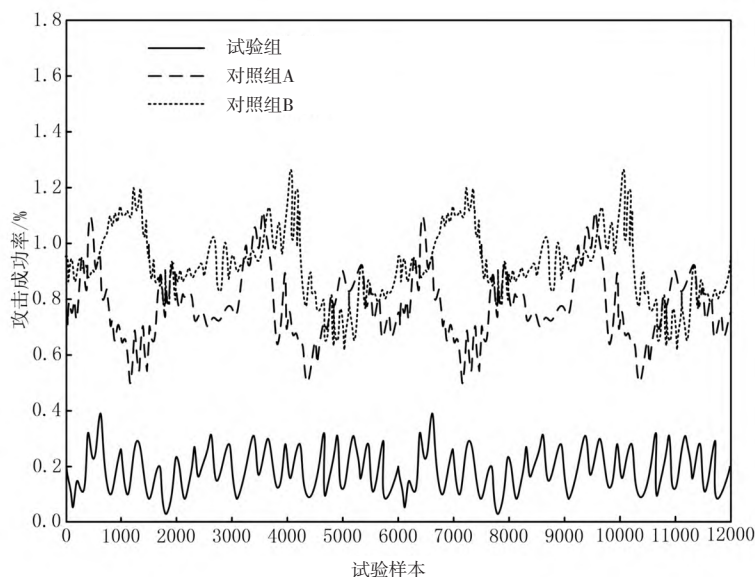


图1 不同身份认证方法的攻击成功率比较

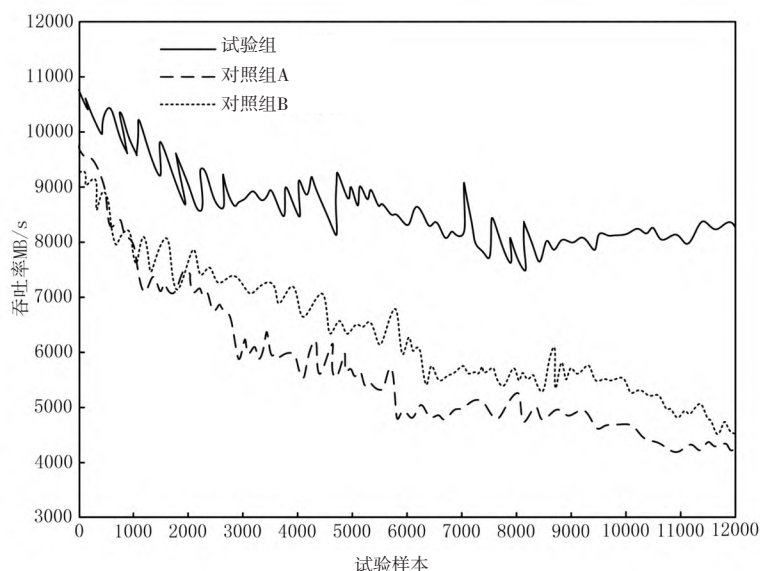


图2 不同身份认证方法的吞吐率比较

方法相比,试验组方法的吞吐率一直处于最高水平,表明改进 MD5 算法处理大量身份认证请求的效率更高。这主要得益于改进 MD5 算法相对简单的计算过程和较低的资源消耗。因此,本文研究的基于改进 MD5 算法的计算机网络用户身份安全认证方法是可行、可靠的,在实际应用中具有更高的安全性与吞吐率。

4 结语

综上所述,本文深入研究了基于改进 MD5 算法的计算机网络用户身份安全认证方法。在研究过程中,采用转换用户身份数据加密参数格式、改进 MD5 算法的消息扩展机制,定义严格的身份认证规则,有效提高了身份认证的安全性和效率。仿真试验结果表明,与传统身份认证方法相比,本文设计方法在面对外界恶意攻击手段过程中具有较高的抗攻击性,同时吞吐率也有显著提升。未来将进一步优化算法实

现,探索更多提高身份认证安全性的方法。

参考文献

- [1] 冉津豪,蔡栋梁.基于区块链和可信执行环境的属性签名身份认证方案[J].计算机研究与发展,2023,60(11):2555-2566.
- [2] 孙敏,李新宇,张鑫.基于多级承诺协议的联盟链身份认证方案研究[J].计算机科学,2024,51(增刊2):854-860.
- [3] 高月红,张雪,李晨阳.基于三次握手与 MD5 的轻量化接入认证算法[J].系统工程与电子技术,2025,47(3):987-996.
- [4] 杨裴裴,朱齐亮.网络链路传输层数据分组加密算法仿真[J].计算机仿真,2023,40(10):390-393,467.
- [5] 张志红,付钰,付伟.中文词义密文模糊搜索算法研究[J].海军工程大学学报,2024,36(6):38-45.